

Sisteme de operare

Laborator 2

Semestrul I 2025-2026

Laborator 2

- file I/O
- descriptori de fisiere
- operatii cu fisiere
- partajarea fisierelor

Descriptori de fisiere

- paradigma de lucru cu fisiere:
 - deschidere/creare fisier
 - citire/scriere
 - inchidere fisier
- fisierele deschise de un proces sunt referite in kernel prin descriptori de fisiere
 - intreg nenegativ intors de apelurile sistem *open/creat*
*int open(const char * pathname, int flags, mode_t mode);*
*int creat(const char * pathname, mode_t mode);*
- functiile de citire/scriere folosesc FDs, nu nume de fisiere !
- conventie Unix: 0,1 & 2 asociati cu stdin (STDIN_FILENO), stdout (STDOUT_FILENO) si stderr (STDERR_FILENO)
 - nu e o proprietate a kernelului, ci a shell-urilor Unix !
 - multe aplicatii Unix vor esua daca nu se respecta conventia

Open/creat/close

- *open*
 - flag-uri de R/W: `O_RDONLY`, `O_WRONLY`, `O_RDWR`
 - alte flag-uri: `O_APPEND`, `O_CREAT`, `O_EXCL`, `O_TRUNC`, `O_NONBLOCK`, `O_SYNC`
 - returneaza (garantat) cel mai mic descriptor de fisiere nealocat
 - folosit de aplicatii pentru a gestiona `stdin/stdout/stderr`
 - ex:


```
close(1);
fd = open("somefile", O_WRONLY); // fd = 1, redirectarea stdout catre "somefile"
```
- *creat*
 - echivalent cu


```
open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode)
```
 - Obs: fisierul e creat doar pt scriere !
 - solutie mai buna:


```
open (pathname, O_RDWR | O_CREAT | O_TRUNC, mode)
```
- *close*

```
int close (int fd);
```

 - cand procesul executa `exit()`, toate fisierele deschise sunt automat inchise cu `close`

Pozitionarea file pointer-ului

- orice fisier deschis are asociat un *offset curent* (referit de *file pointer*)
 - nr. nenegativ care exprima pozitia curenta (exprimata in octeti) de la care se poate citi sau la care se poate scrie din sau in fisier
 - operatiile de citire/scriere folosesc implicit acest offset si incrementeaza corespunzator offsetul curent
 - initializat cu 0 daca fisierul nu a fost deschis cu O_APPEND
- pozitionarea explicita a file pointer-ului (FP):

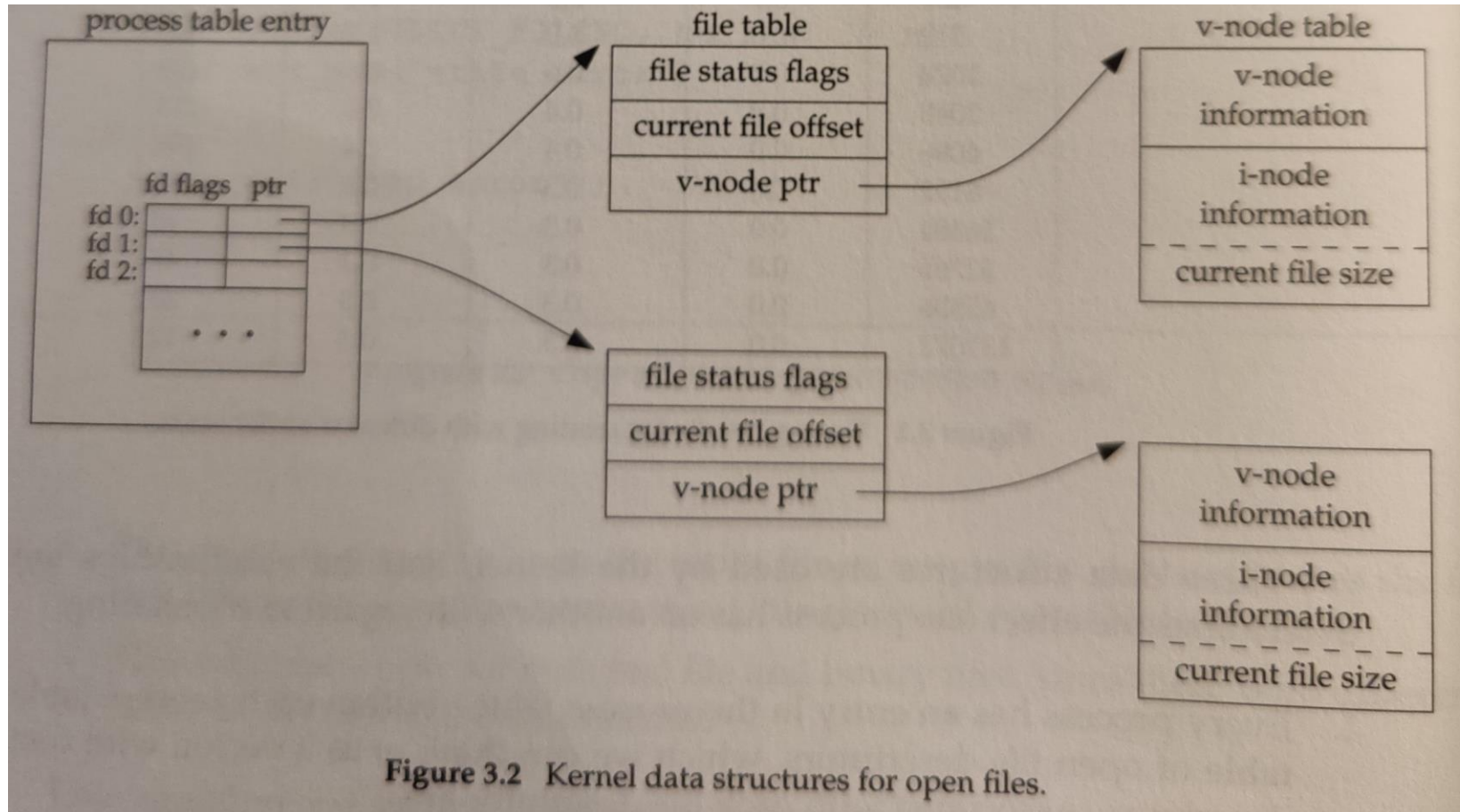
off_t lseek(int fd, off_t offset, int whence)

- valoarea parametrului *offset* este relativa la valoarea parametrului *whence* care poate fi
 - SEEK_SET: FP este pozitionat la *offset* bytes de la inceputul fisierului
 - SEEK_CUR: FP este incrementat/decrementat cu *offset* bytes fata de offsetul curent
 - SEEK_END: FP este pozitionat relativ la sfarsitul fisierului (*offset* poate fi pozitiv sau negativ)
- intoarce valoarea noului offset curent (file pointer)
- poate fi folosita pentru a testa daca fisierul referit este capabil de *seek*
 - Eg., *lseek* cu pipe-uri, stdin intoarce eroare

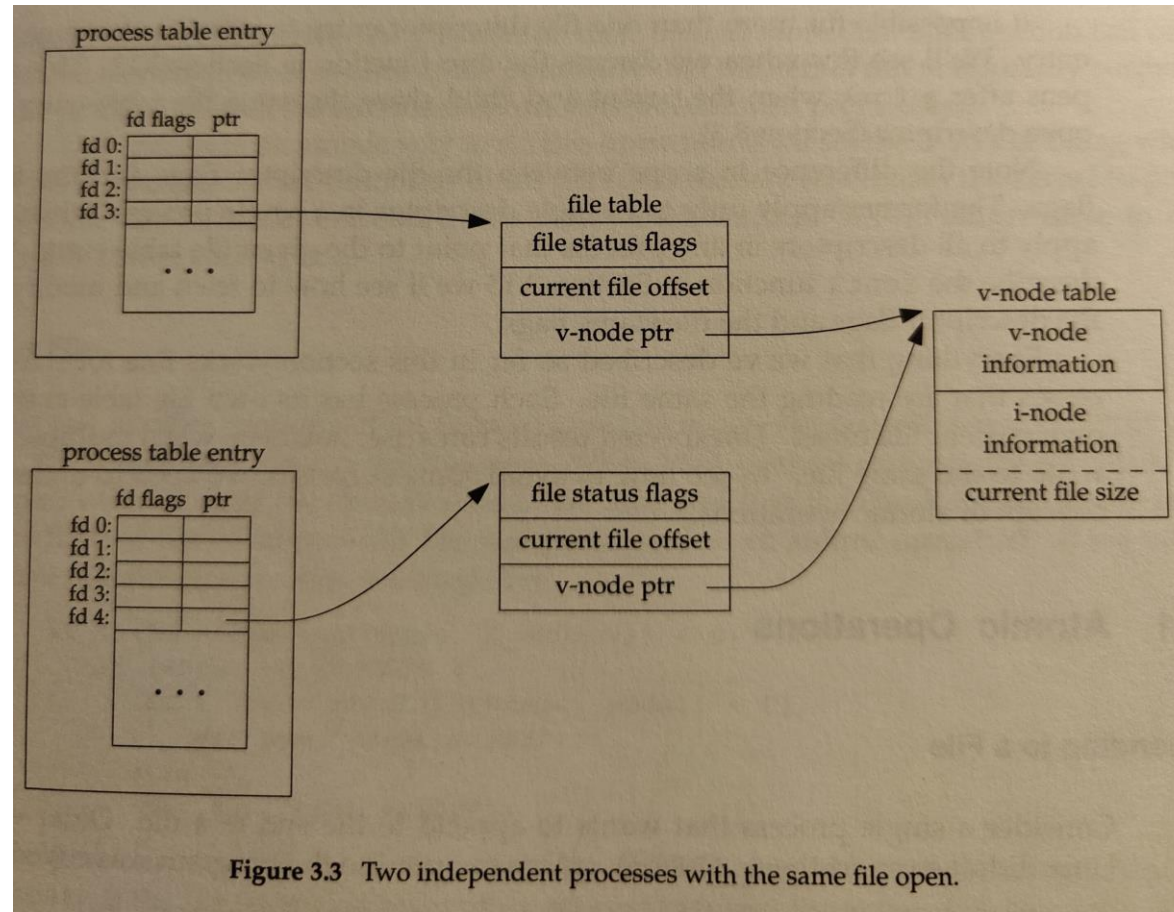
Read/write

- *ssize_t read(int fd, void *buf, size_t count);*
 - intoarce nr de octeti cititi (0 daca s-a ajuns la EOF) in *buf*
 - daca FP + count > EOF, intoarce nr de octeti disponibili
 - cand *fd* refera un terminal, se citeste cate o linie (eg, pana la $\backslash n$)
 - cand *fd* este un socket (citire din retea) e posibil sa intoarca mai putin octeti decat *count*
 - citeste incepand de la offsetul curent si incrementeaza FP cu *count* octeti
- *ssize_t write(int fd, void *buf, size_t count);*
 - intoace nr de octeti scrisi
 - scrie *count* octeti incepand de la offsetul current
 - incrementeaza FP cu valoarea *count* (sau cu nr de octeti care au putut fi scrisi cu succes, v. situatia de disc plin)
 - pt. fisiere deschise in mod O_APPEND, FP se pozitioneaza la sf. fisierului si de acolo incepe scrierea

Structuri de date pt fisiere deschise



Partajarea fisierelor



Operatii atomice

- ex: operatia *append* executata de doua procese A si B
- fiecare proces executa:

lseek(fd, 0, SEEK_END);

write(fd, buf, 100);

- scenariu:
 - A executa *lseek* si pierde procesorul
 - B executa *lseek + write* si extinde fisierul
 - cand A revine pe procesor si executa *write* va suprascrie ceea ce a scris B
- solutie: A si B deschid fisierul cu O_APPEND
- NB: *operatiile care necesita apeluri multiple NU SUNT ATOMICE !*
O solutie corecta necesita mijloace specifice de sincronizare

Fcntl/dup/dup2

- *fcntl*

- schimba proprietatile unui fisier deschis

int fcntl(int fd, int cmd, ... / args */);*

- *cmd*:

- F_DUPFD: duplica descriptorul de fisier din *args*, *fcntl* intoarce noul fd (cea mai mica valoare nealocata $\geq$ argumentul din *args*)
- F_GETFD/F_SETFD: get/set flag-uri fd (FD_CLOEXEC)
- F_GETFL/F_SETFL: get/set status flags pt fd (O_RDONLY, O_WRONLY, O_RDWR, O_APPEND, O_NONBLOCK, O_SYNC, O_ASYNC)
- F_GETLK/F_SETLK: get/set record locks

- *dup/dup2*

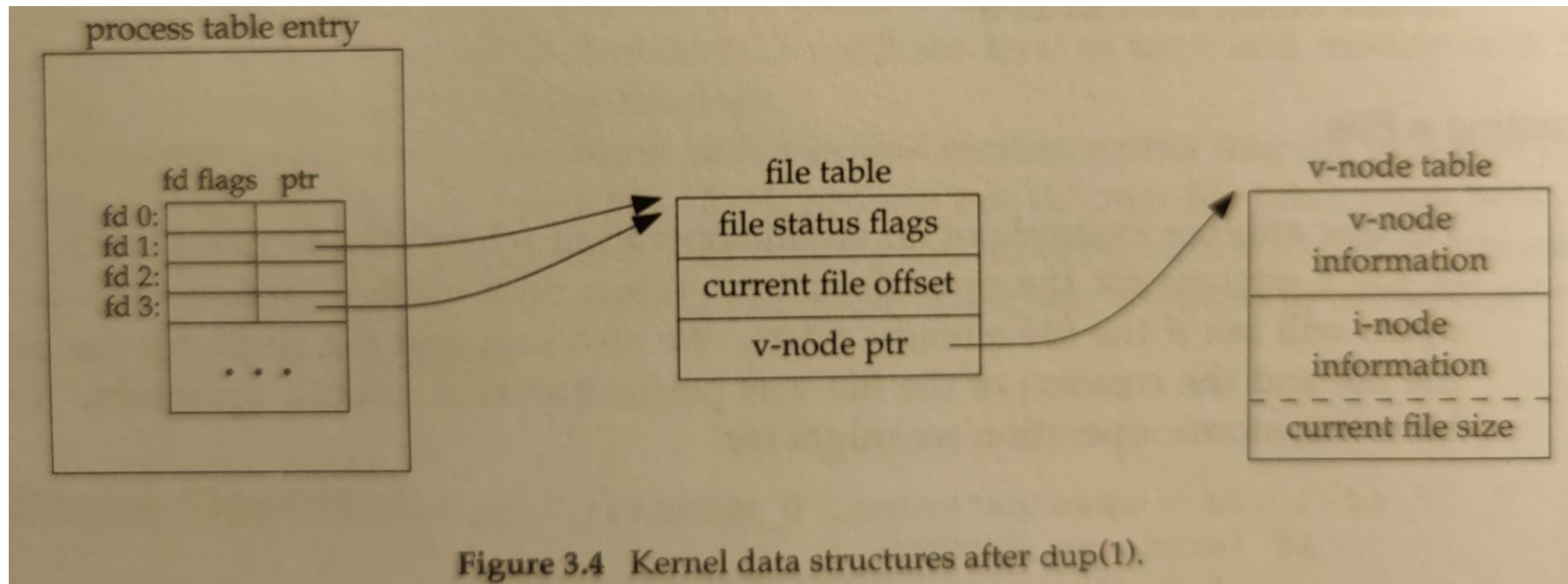
newfd = dup(fd) $\Leftrightarrow$ *fcntl(fd, F_DUPFD, 0)*

dup2(fd, newfd) $\Leftrightarrow$ *close(newfd);*

fcntl(fd, F_DUPFD, newfd);

Obs: *dup2* e op. atomica ! *Close* + *fcntl* NU !

Dup



Sisteme de operare

Laborator 2

File I/O

1. Creati un nou subdirector *lab2/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc*, *src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab2/bin*.

2. Scrieti un program C **mycp.c** care primeste doi parametri in linia de comanda: primul parametru este numele unui fisier care trebuie copiat intr-un alt fisier al carui nume este furnizat de al doilea parametru, ca mai jos:

```
$ mycp fisier-src fisier-dst
```

Comparati programul **mycp.c** cu programul **mycp.c** pe care l-ati scris in laboratorul trecut si care copia in bucla date de la file descriptorul 0 si le scria pe file descriptorul 1. Ce observati?

3. Modificati programul **mycp.c** din laboratorul 1, care citeste in bucla date de la *stdin* (file descriptor 0) si le scrie la *stdout* (file descriptor 1) a.i. va primi doua argumente care reprezinta numele fisierului sursa care trebuie copiat si numele fisierului destinatie. Fara sa modificati deloc bucla de copiere (adica pastrati file descriptorii 0 si 1 folositi), deschideti corespunzator cele doua fisiere primite ca argumente si folosind *dup/dup2* faceti programul sa functioneze ca un program de copiere de fisiere ca la punctul 2.

```
$ mycp fisier-sursa fisier-destinatie
```

Ce se intampla daca fisierul sursa este chiar terminalul pe care lucram, obtinut cu comanda *tty*? De exemplu:

```
$ mycp `tty` fisier-destinatie
```

4. Modificati programul **mycp.c** de la punctul 2 a.i. dupa ce deschide fisierul sursa, programul seteaza file descriptorul obtinut pentru acces non-blocant cu ajutorul apelului sistem *fcntl*. Care e diferenta dintre executia celor doua apeluri ale comenzii din secventa de comenzi de mai jos? Cum va explicati rezultatul?

```
$ mycp fisier-sursa fisier-destinatie  
$ rm fisier-destinatie  
$ mycp `tty` fisier-destinatie
```

5. Scrieti un program C **hole.c** care creaza un fisier cu “goluri/gauri” in el. De pilda, dupa ce ati creat fisierul, scrieti un sir de caractere la inceputul fisierului, mutati offsetul curent din fisier la valoarea 100 si scrieti un alt sir de caractere acolo. Care este dimensiunea fisierului nou creat in octeti afisata de comenzi cum ar fi *ls -l <fisier_cu_goluri>* sau *du -sb <fisier_cu_goluri>*?

Dar comanda *od -c <fisier_cu_goluri>* ce va arata?

6. Scrieti un program C **true-du.c** care primeste un fisier ca parametru si afiseaza dimensiunea lui in octeti fara goluri (adica dimensiunea sa reala, fara goluri, avand in vedere ca acestea nu se stocheaza pe disc).